

net-08 - Packet Walkthrough

Contexto

Este treinamento oferece um contexto mais avançado e em mais detalhes do funcionamento dos diferentes tipos de conectividade no Overlay. Faremos um Packet Walkthrough para entender os fluxos de comunicação abordando:

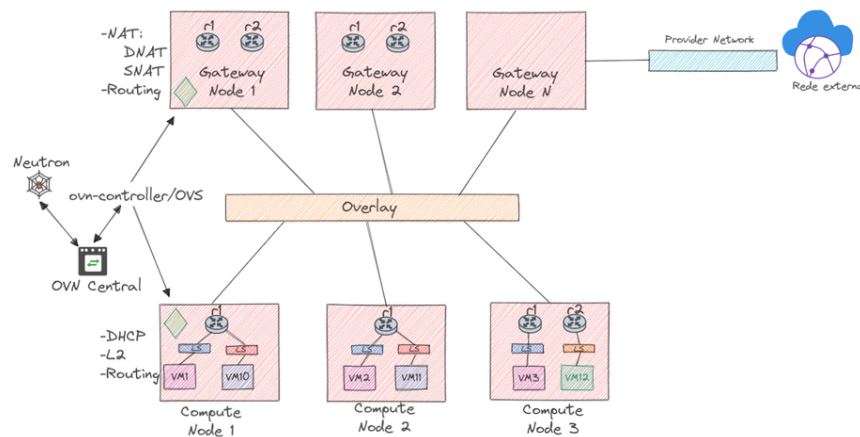
- contexto geral da arquitetura
- DHCP
- conectividade leste-oeste
- conectividade norte-sul
- Openflow Walkthrough

Esse treinamento utiliza o setup do treinamento 07

Contexto geral da arquitetura

Antes de entrar em mais detalhes no funcionamento dos diferentes tipos de conectividade no Overlay, iremos recapitular alguns pontos importantes dentro de nossa arquitetura.

O diagrama abaixo tenta resumir as diferentes camadas e funções de nossa arquitetura:



- **Neutron** - Atua como CMS para criação das topologias de rede
- **OVN Central** - SDN Controller
- **Gateway Nodes** - Fazem o papel de conectar os workloads da plataforma de Cloud com o mundo externo, provendo ou não o serviço de NAT e de roteamento
- **Compute Nodes** - Além de virtualizar os workloads, são responsáveis pela conectividade leste-oeste e norte-sul contra os Gateway Nodes, provendo os serviços de L2, DHCP e roteamento interno
- **Chassis** - Gateway Node e Compute Nodes executando o ovn-controller e OVS
- **Logical Routers(LR)** - São distribuídos entre os Chassis de acordo com o instanciamento dos workloads e são responsáveis pelo roteamento e NAT
- **Logical Switch(LS)** - São distribuídos entre os Chassis para prover conectividade L2 e conectam as Logical Router Ports(LRP) dos Logical Routers e as Logical Switch Ports(LSP) com os workloads

Packet Walkthrough - DHCP

Nesse tópico faremos uma análise do tráfego de DHCP de um workload e entenderemos o fluxo completo desde o Northbound até os Physical Flows no Datapath.

Tudo começa a partir do Neutron que irá configurar o serviço de DHCP com alguns parâmetros, por padrão ao criar uma subnet você pode determinar o range de endereço de DHCP, os endereços IP de DNS server e dentre outras opções.

Vamos analisar a configuração criada automaticamente pelo Neutron no OVN Northbound. A network **private** e a **vm1** serão nossos alvos:

```
1 $ openstack network list | grep private
2 | 5bb5b9ad-7348-474f-9e14-66f34278e373 | private | 229329b1-1627-49e2-bfb3-deacba7155f9, f3a990c6-e32b-4746-9983-fb5bf42020c4 |
3 $ openstack server list | grep vm1
4 | dac8d874-1aa5-494c-88a6-234fd1575201 | vm1 | ACTIVE | private=10.0.0.42, 172.24.4.96, fd42:4193:f777:0:f816:3eff:fe77:1296 | cirros-0.5.2-x86_64-disk
5 $ openstack port list | grep 10.0.0.42
6 | 04a459b1-6ef0-45c2-aedc-063c63b716f9 | | fa:16:3e:77:12:96 | ip_address='10.0.0.42', subnet_id='229329b1-1627-49e2-bfb3-deacba7155f9'
```

Com o output acima, encontre o ID da subnet da network **private** (ex: **229329b1-1627-49e2-bfb3-deacba7155f9**). Com essa informação em mãos, podemos analisar as configurações criadas em formato de Logical Flows:

```
1 # ovn-nbctl find dhcp_options "external_ids:\\"subnet_id\\"=\""229329b1-1627-49e2-bfb3-deacba7155f9"
2 _uuid : 1c0551b3-ad8f-44b2-92d5-9d457b4c2d69
3 cidr : "10.0.0.0/26"
```

```

4 external_ids      : {"neutron:revision_number"="0", subnet_id="229329b1-1627-49e2-bfb3-deacba7155f9"}
5 options          : {classless_static_route="{169.254.169.254/32,10.0.0.2, 0.0.0.0/0,10.0.0.1}", dns_server="{8.8.8.8}", domain_name="\openstackgate

```

Do ponto de vista do OVN Southbound, podemos verificar os flows que serão instalados nos ovn-controller. Antes de mais nada, verifique o ID da network **private**(ex: 5bb5b9ad-7348-474f-9e14-66f34278e373) para determinarmos o Datapath do Logical Switch:

```

1 # ovn-nbctl show | grep 5bb5b9ad-7348-474f-9e14-66f34278e373
2 switch 4830c62f-7de6-4c6d-91aa-6eade80c7291 (neutron-5bb5b9ad-7348-474f-9e14-66f34278e373) (aka private)
3 # ovn-sbctl lflow-list 4830c62f-7de6-4c6d-91aa-6eade80c7291 | grep dhcp
4 table=19(ls_in_dhcp_options ), priority=100 , match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4.src == 0.
5 table=19(ls_in_dhcp_options ), priority=100 , match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4.src == 16
6 table=19(ls_in_dhcp_options ), priority=0 , match=(1), action=(next;)
7 table=20(ls_in_dhcp_response), priority=100 , match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4 && udp.sr
8 table=20(ls_in_dhcp_response), priority=0 , match=(1), action=(next;)

```

Basicamente, o **OVN-Northd** adicionou um Logical Flow de DHCP offer e reply para cada Logical Switch Port com **ls_in_dhcp_options** e **ls_in_dhcp_response**. A action **put_dhcp_opts** transforma o pacote de DHCP request em um pacote de DHCP reply.

No nível do ovs-vswitchd, podemos verificar os Openflow flows instalados para o MAC Address de nossa VM:

```

1 # ovs-ofctl dump-flows br-int | grep fa:16:3e:77:12:96
2
3 cookie=0xc8a2b498, duration=357750.527s, table=8, n_packets=350012, n_bytes=33758335, idle_age=71, hard_age=65534, priority=50, reg14=0x4, metadata=0x1,
4 cookie=0xbdf0e570, duration=357750.536s, table=9, n_packets=918, n_bytes=78868, idle_age=71, hard_age=65534, priority=90, ipv6, reg14=0x4, metadata=0x1,
5 cookie=0xbdf0e570, duration=357750.528s, table=9, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, ipv6, reg14=0x4, metadata=0x1, dl
6 cookie=0xabeb0292, duration=357750.535s, table=9, n_packets=339374, n_bytes=33263385, idle_age=10317, hard_age=65534, priority=90, ip, reg14=0x4, metada
7 cookie=0xe9330c27, duration=357750.531s, table=9, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x1, dl
8 cookie=0xe9330c27, duration=357750.531s, table=9, n_packets=20, n_bytes=1720, idle_age=10307, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x
9 cookie=0xe9330c27, duration=357750.531s, table=9, n_packets=20, n_bytes=1800, idle_age=10309, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x
10 cookie=0xd65702f, duration=357750.531s, table=9, n_packets=20, n_bytes=6842, idle_age=10309, hard_age=65534, priority=90, udp, reg14=0x4, metadata=0x1, d
11 cookie=0x330e8e71, duration=357750.531s, table=9, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=80, ipv6, reg14=0x4, metadata=0x1, dl
12 cookie=0x330e8e71, duration=357750.527s, table=9, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=80, ip, reg14=0x4, metadata=0x1, dl_s
13 cookie=0x9dbc8824, duration=357750.536s, table=10, n_packets=20, n_bytes=1720, idle_age=10307, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0
14 cookie=0x9dbc8824, duration=357750.531s, table=10, n_packets=888, n_bytes=76368, idle_age=71, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x
15 cookie=0x0, duration=357750.531s, table=10, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x1, dl_src=f
16 cookie=0x0, duration=357750.531s, table=10, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x1, dl_src=f
17 cookie=0xf9d07571, duration=357750.528s, table=10, n_packets=9660, n_bytes=405720, idle_age=10317, hard_age=65534, priority=90, arp, reg14=0x4, metadate
18 cookie=0x9dbc8824, duration=357750.528s, table=10, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, conj_id=3013700774, icmp6, reg14
19 cookie=0x0, duration=357750.528s, table=10, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x1, dl_src=f
20 cookie=0x0, duration=357750.527s, table=10, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, icmp6, reg14=0x4, metadata=0x1, dl_src=f
21 cookie=0x5ad678bb, duration=357750.529s, table=23, n_packets=6, n_bytes=588, idle_age=65534, hard_age=65534, priority=100, reg0=0xa00002a, reg15=0x1, me
22 cookie=0x16831f3e, duration=357750.528s, table=23, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=100, reg0=0xfd424193, reg1=0xf7770e
23 cookie=0x20e5146e, duration=357750.528s, table=26, n_packets=1, n_bytes=42, idle_age=65534, hard_age=65534, priority=50, arp, metadata=0x1, arp_tpa=10.0
24 cookie=0x38339328, duration=357750.537s, table=27, n_packets=20, n_bytes=6842, idle_age=10309, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x
25 cookie=0x7bf635c5, duration=357750.532s, table=27, n_packets=24, n_bytes=8208, idle_age=10317, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x
26 cookie=0x7bf635c5, duration=357750.532s, table=27, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x1, dl
27 cookie=0x86ea486d, duration=357750.532s, table=28, n_packets=35, n_bytes=12775, idle_age=10309, hard_age=65534, priority=100, udp, reg0=0x8/0x8, reg14=0
28 cookie=0x6de084c9, duration=357750.529s, table=32, n_packets=62, n_bytes=7092, idle_age=10308, hard_age=65534, priority=50, metadata=0x1, dl_dst=fa:16:
29 cookie=0xe2ed6c32, duration=357750.529s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, ip, reg15=0x4, metadata=0x1, dl_c
30 cookie=0xe2ed6c32, duration=357750.529s, table=48, n_packets=339324, n_bytes=33263913, idle_age=10309, hard_age=65534, priority=90, ip, reg15=0x4, metad
31 cookie=0xe2ed6c32, duration=357750.528s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, ip, reg15=0x4, metadata=0x1, dl_c
32 cookie=0x2e59736, duration=357750.529s, table=48, n_packets=10, n_bytes=1180, idle_age=10308, hard_age=65534, priority=90, ipv6, reg15=0x4, metadata=0x1, dl
33 cookie=0x2e59736, duration=357750.529s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, ipv6, reg15=0x4, metadata=0x1, dl
34 cookie=0x2e59736, duration=357750.528s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=90, ipv6, reg15=0x4, metadata=0x1, dl
35 cookie=0xab65b467, duration=357750.532s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=80, ipv6, reg15=0x4, metadata=0x1, dl
36 cookie=0xab65b467, duration=357750.529s, table=48, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=80, ip, reg15=0x4, metadata=0x1, dl_c
37 cookie=0xad097abf, duration=357750.532s, table=49, n_packets=348994, n_bytes=33670813, idle_age=10308, hard_age=65534, priority=50, reg15=0x4, metadata

```

Vamos realizar um filtro por flows que tenham relação com DHCP:

```

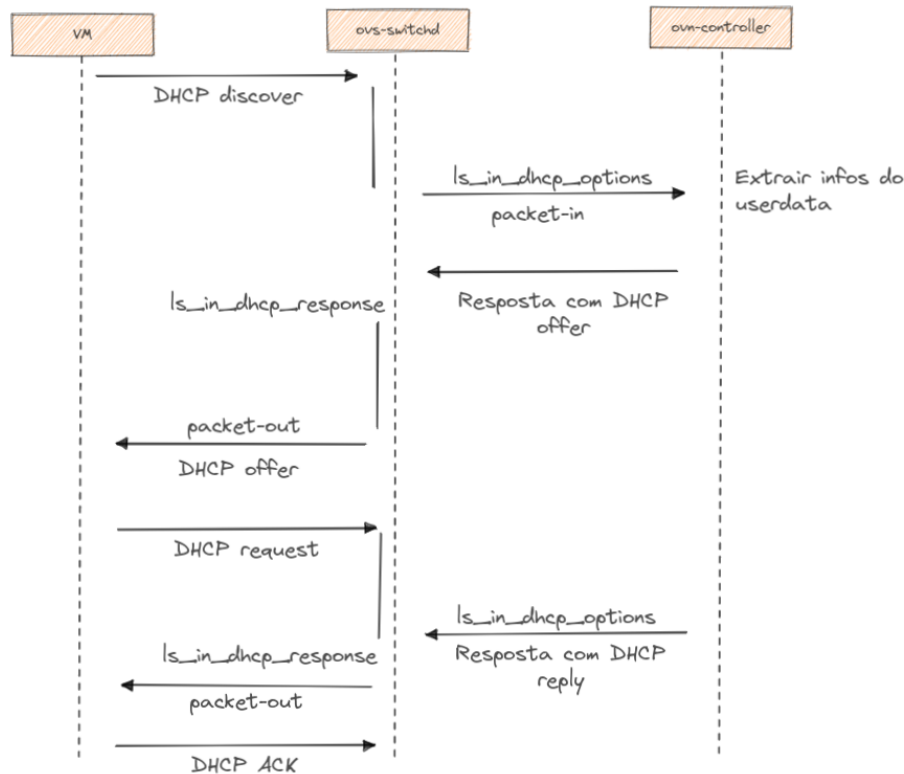
1 # ovs-ofctl dump-flows br-int | grep fa:16:3e:77:12:96 | grep 68
2 cookie=0xd65702f, duration=357793.538s, table=9, n_packets=20, n_bytes=6842, idle_age=10352, hard_age=65534, priority=90, udp, reg14=0x4, metadata=0x1, dl
3 cookie=0x38339328, duration=357793.545s, table=27, n_packets=20, n_bytes=6842, idle_age=10352, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x1,
4 cookie=0x7bf635c5, duration=357793.540s, table=27, n_packets=24, n_bytes=8208, idle_age=10360, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x1,
5 cookie=0x7bf635c5, duration=357793.540s, table=27, n_packets=0, n_bytes=0, idle_age=65534, hard_age=65534, priority=100, udp, reg14=0x4, metadata=0x1, dl_s
6 cookie=0x86ea486d, duration=357793.540s, table=28, n_packets=35, n_bytes=12775, idle_age=10352, hard_age=65534, priority=100, udp, reg0=0x8/0x8, reg14=0x4

```

Ao verificar o output acima, a action **put_dhcp_opts** está sendo traduzida para a action **controller** com a flag **pause** que irá configurar o **dhcp options** presente no campo userdata.

Quando um pacote de DHCP offer/request é recebido, o **ovs-vswitchd** irá enviar o mesmo para o **ovn-controller** que irá verificar o **offer IP** e o **dhcp options** presentes no campo userdata e após fazer esse processamento, devolverá a resposta para o **ovs-switchd** que retornará com o DHCP offer/reply.

Abaixo podemos avaliar o diagrama com a sequência de ações que ocorrem quando uma VM envia um DHCP discover/request:



No nível dos Physical Flows/Datapath, podemos realizar uma análise do processo para obter um endereço de IP via DHCP:

Na vm1, desabilite a interface de rede:

```
1 # ifdown eth0
```

Com essa ação, o processo de DHCP release é invocado e pode ser observado no log do ovn-controller e também nos flows que serão processados no nível do Datapath:

```

1 # tail -f /var/log/ovn/ovn-controller.log
2 2022-06-13T14:07:28.133Z|00045|pinctrl(ovn_pinctrl0)|INFO|DHCPRELEASE fa:16:3e:77:12:96 10.0.0.42
3
4 # ovs-dpctl dump-flows | grep fa:16:3e:77:12:96
5 recirc_id(0),in_port(4),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e:8f:c6:8b),eth_type(0x0800),ipv4(src=10.0.0.42,proto=17,frag=no),udp(src=0/0xfe00),packe
6 recirc_id(0),in_port(4),ct_state(-new-est-rel-rpl-inv-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=ff:ff:ff:ff:ff:ff),eth_type(0x0806),arp(sip=10
7 recirc_id(0xb2),in_port(4),ct_state(+new-est-rel-rpl-inv+trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=fa:16:3e:8f:c6:8b),eth_type(0x0800),ipv4(sr

```

Como observado acima, o Datapath (fast path) enviou uma consulta para o ovs-vsitchd (slow path) que executou um processo similar ao do DHCP request porém processando o DHCP release.

Continuando com nossa análise, na vm1 habilite a interface de rede:

```

1 # ifup eth0
2 udhcpd: started, v1.29.3
3 udhcpd: sending discover
4 udhcpd: sending select for 10.0.0.42
5 udhcpd: lease of 10.0.0.42 obtained, lease time 43200
6 route: SIOCADDRT: File exists
7 WARN: failed: route add -net "0.0.0.0/0" gw "10.0.0.1"

```

Com essa ação, o processo de DHCP discover/request é invocado e pode ser observado no log do ovn-controller e também nos flows que serão processados no nível do Datapath:

```

1 # tail -f /var/log/ovn/ovn-controller.log
2 2022-06-13T14:07:35.868Z|00046|pinctrl(ovn_pinctrl0)|INFO|DHCP OFFER fa:16:3e:77:12:96 10.0.0.42
3 2022-06-13T14:07:35.874Z|00047|pinctrl(ovn_pinctrl0)|INFO|DHCPACK fa:16:3e:77:12:96 10.0.0.42
4
5 # ovs-dpctl dump-flows | grep fa:16:3e:77:12:96
6 recirc_id(0),in_port(4),ct_state(-new-est-rpl-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=33:33:00:00:00:16),eth_type(0x86dd),ipv6(src=:,dst=f
7 recirc_id(0),in_port(4),ct_state(-new-est-rpl-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=33:33:ff:77:12:96),eth_type(0x86dd),ipv6(src=:,dst=f
8 recirc_id(0),in_port(4),ct_state(-new-est-rpl-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=33:33:00:00:00:02),eth_type(0x86dd),ipv6(src=fe80::fe
9 recirc_id(0),in_port(4),ct_state(-new-est-rpl-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=33:33:ff:77:12:96),eth_type(0x86dd),ipv6(src=:,dst=f
10 recirc_id(0),in_port(4),ct_state(-new-est-rel-rpl-inv-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=ff:ff:ff:ff:ff:ff),eth_type(0x0800),ipv4(src=
11 recirc_id(0),in_port(4),ct_state(-new-est-rpl-trk),ct_label(0/0x1),eth(src=fa:16:3e:77:12:96,dst=33:33:00:00:00:16),eth_type(0x86dd),ipv6(src=fe80::fe

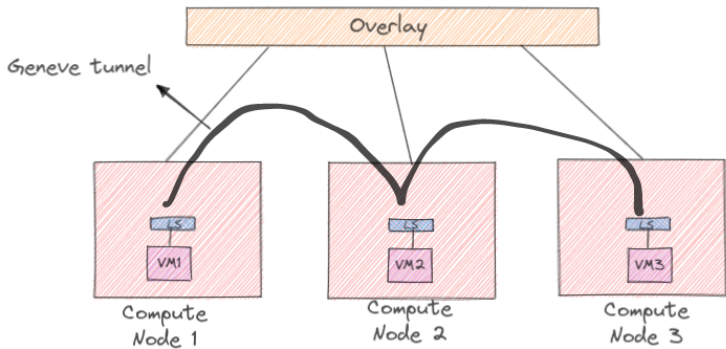
```

Packet Walkthrough - Conectividade leste-oeste

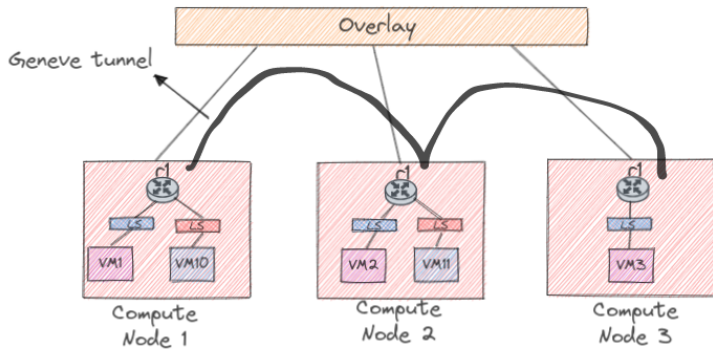
A conectividade leste-oeste se resume na comunicação de rede entre workloads que estão na mesma localidade.

Existem alguns tipos de conectividade leste-oeste que podemos destacar:

- entre workloads que estão em uma mesma subnet
 - workloads no mesmo compute node (Chassi)
 - workloads em diferentes compute nodes (Chassi)



- entre workloads que estão em diferentes subnets porém abaixo do mesmo router
 - workloads no mesmo compute node (Chassi)
 - workloads em diferentes compute nodes (Chassi)



Do ponto de vista do OVN Southbound database, podemos verificar entradas nas diversas tables e determinar um link lógico entre o que foi definido no OVN Northbound database com sua localização na infraestrutura(compute nodes).

Na table **Port_Binding** podemos visualizar todos os tipos de portas presentes e sua localização na infraestrutura:

```
1 # ovn-sbctl list port_binding
```

Vamos avaliar alguns tipos de portas:

1	_uuid	: f3be1b3d-588b-412c-a495-40a2a2fb84ea
2	chassis	: []
3	datapath	: 4f59f6dd-5745-4634-8cd4-52af21720b31
4	encap	: []
5	external_ids	: {"neutron:network_name"=neutron-5bb5b9ad-7348-474f-9e14-66f34278e373, "neutron:revision_number"="3", "neutron:router_name"="d186
6	gateway_chassis	: []
7	ha_chassis_group	: []
8	logical_port	: lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
9	mac	: ["fa:16:3e:8f:c6:8b 10.0.0.1/26"]
10	nat_addresses	: []
11	options	: {peer="286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"}
12	parent_port	: []
13	requested_chassis	: []
14	tag	: []
15	tunnel_key	: 1
16	type	: patch
17	up	: false
18	virtual_parent	: []

No output acima, temos uma **Logical Router port** (Gateway de uma subnet) do tipo **patch port** e que é distribuída por vários Chassis onde temos VMs conectadas em um Logical Switch em específico.

```
1  _uuid          : a9906c02-8e09-4101-a5d9-b4765308db43
2  chassis        : []
3  datapath       : 89f1e932-67a5-4868-8008-e3fd3ef11816
4  encap          : []
5  external_ids   : {"neutron:cidrs"="10.0.0.2/26 fd42:4193:f777:0:f816:3eff:fe3d:c69e/64", "neutron:device_id"="ovnmeta-5bb5b9ad-7348-474f-9e14-66f34278e373"}
6  gateway_chassis : []
7  ha_chassis_group : []
8  logical_port    : "7b399866-1eb0-4654-9f7b-04240ec12e2a"
9  mac            : ["fa:16:3e:3d:c6:9e 10.0.0.2 fd42:4193:f777:0:f816:3eff:fe3d:c69e"]
10 nat_addresses  : []
11 options        : {"requested-chassis":""}
12 parent_port     : []
13 requested_chassis : []
14 tag            : []
15 tunnel_key      : 1
16 type           : localport
17 up             : false
18 virtual_parent  : []
```

No output acima, temos uma porta do tipo **localport** que é para o serviço de metadata e que também é distribuída por diversos Chassis que tenham uma VM em um Logical Switch em específico.

```
1  _uuid          : ae5a82a2-b438-4daf-a74e-320e9683699d
2  chassis        : b283613e-8805-4bc0-a757-3c4897f9349f
3  datapath       : 89f1e932-67a5-4868-8008-e3fd3ef11816
4  encap          : []
5  external_ids   : {"neutron:cidrs"="10.0.0.42/26 fd42:4193:f777:0:f816:3eff:fe77:1296/64", "neutron:device_id"="dac8d874-1aa5-494c-88a6-234fd1575200"}
6  gateway_chassis : []
7  ha_chassis_group : []
8  logical_port    : "04a459b1-6ef0-45c2-aedc-063c63b716f9"
9  mac            : ["fa:16:3e:77:12:96 10.0.0.42 fd42:4193:f777:0:f816:3eff:fe77:1296"]
10 nat_addresses  : []
11 options        : {"mcast_flood_reports"="true", requested-chassis=ovn-sandbox1}
12 parent_port     : []
13 requested_chassis : b283613e-8805-4bc0-a757-3c4897f9349f
14 tag            : []
15 tunnel_key      : 4
16 type           : ""
17 up             : true
18 virtual_parent  : []
```

No output acima, temos uma **Logical Switch Port** que conecta uma VM em um Logical Switch. Ela acaba existindo somente no Chassi em que a VM está em execução e possui o ID do Chassi associado. O campo `tunnel_key` define o ID da porta dentro de um túnel Geneve com um Chassi.

Podemos verificar o ID do Chassi onde essa VM possui a Logical Switch port:

```
1  # ovn-sbctl list chassis b283613e-8805-4bc0-a757-3c4897f9349f
2  _uuid          : b283613e-8805-4bc0-a757-3c4897f9349f
3  encaps         : [3c5de98a-4e23-4eb8-b8b1-95b65c980692]
4  external_ids   : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", interface-id=0}
5  hostname       : ovn-sandbox1
6  name           : "cab023cb-ba95-4ec3-aa9b-61c2267833cd"
7  nb_cfg         : 0
8  other_config   : {datapath-type=system, iface-types="bareudp,erspan,geneve,gre,gtpu,internal,ip6erspan,ip6gre,lisp,patch,stt,system,tap,vxlan", interface-id=0}
9  transport_zones : []
10 vtep_logical_switches: []
```

Baseado nas portas lógicas instanciadas em cada Chassi, serão instalados os Openflow flows para que os pipelines de Ingress e Egress sejam processados para cada pacote.

Podemos avaliar os Logical Flows que serão traduzidos para Openflow flows por Logical Switch, primeiramente podemos verificar as Logical Switch Ports conectadas em uma network (Logical Switch):

```
1  # ovn-nbctl show | grep 5bb5b9ad-7348-474f-9e14-66f34278e373
2  switch 4830c62f-7de6-4c6d-91aa-6eade80c7291 (neutron-5bb5b9ad-7348-474f-9e14-66f34278e373) (aka private)
3
4  # ovn-nbctl show 4830c62f-7de6-4c6d-91aa-6eade80c7291
5  switch 4830c62f-7de6-4c6d-91aa-6eade80c7291 (neutron-5bb5b9ad-7348-474f-9e14-66f34278e373) (aka private)
6      port 286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
7          type: router
8          router-port: lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
9      port 7b399866-1eb0-4654-9f7b-04240ec12e2a
10         type: localport
11         addresses: ["fa:16:3e:3d:c6:9e 10.0.0.2 fd42:4193:f777:0:f816:3eff:fe3d:c69e"]
12      port 04a459b1-6ef0-45c2-aedc-063c63b716f9
```

```

13     addresses: ["fa:16:3e:77:12:96 10.0.0.42 fd42:4193:f777:0:f816:3eff:fe77:1296"]
14     port 7bbb8967-b1dd-475e-b9ec-e6aba3931183
15     type: router
16     router-port: lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
17     port 7e5871ed-6dd5-46bf-a30d-b02dfc3981dc
18     addresses: ["fa:16:3e:5f:33:3f 10.0.0.7 fd42:4193:f777:0:f816:3eff:fe5f:333f"]

```

Faremos uma análise do tráfego L2 entre duas VMs em um mesmo Logical Switch e entenderemos os Logical Flows instalados nos pipelines de Ingress e Egress.

É possível checar os Logical Flows de um determinado Logical Switch conforme abaixo:

```

1 # ovn-sbctl lflow-list 4830c62f-7de6-4c6d-91aa-6eade80c7291

```

Iremos omitir o output por ter uma saída extensa e focaremos nos principais estágios dos pipelines.

O Logical Switch possui um pipeline de Ingress e outro de Egress e a direção é da perspectiva do Logical Switch. O último objetivo do pipeline de Ingress é determinar a porta de output e enviar o pacote para o pipeline de Egress. Sendo que o pipeline de Egress irá realizar algumas validações de segurança e irá entregar o pacote para seu destino.

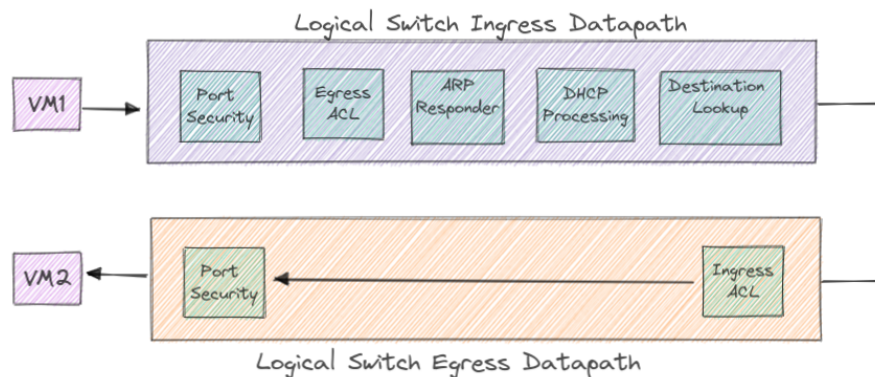
Dois pontos são importantes para entender sobre os pipelines:

- Ambos podem estar localizados no mesmo ou em diferentes Chassis sendo que no caso de estarem em diferentes Chassis, o pacote será enviado por um túnel Geneve
- O pipeline de Egress não possui o estágio de Destination Lookup, ou seja, o que exige que a porta de output precise ser determinada no pipeline de Ingress. Isso significa que o Destination Lookup não precisa ser executado duas vezes.

Usaremos nesse exemplo as seguintes informações de cada VM:

- **VM1**
 - Switch Logical Port: 04a459b1-6ef0-45c2-aedc-063c63b716f9
 - MAC Address: fa:16:3e:77:12:96
 - IP Address: 10.0.0.42
- **VM2**
 - Switch Logical Port: 7e5871ed-6dd5-46bf-a30d-b02dfc3981dc
 - MAC Address: fa:16:3e:5f:33:3f
 - IP Address: 10.0.0.7

Podemos verificar o resumo dos estágios de cada pipeline no diagrama abaixo:



Port Security - Verifica se os pacotes que estão ingressando na Switch Logical Port possuem os endereços de MAC Address e IP de origem corretos:

```

1 table=0 (ls_in_port_sec_l2),priority=50,match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == {fa:16:3e:77:12:96}), action=(next;)
2 table=0 (ls_in_port_sec_l2),priority=50,match=(inport == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.src == {fa:16:3e:5f:33:3f}), action=(next;)
3 table=1 (ls_in_port_sec_ip),priority=90,match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4.src == {10.0.0.42}), action=(next;)
4 table=1 (ls_in_port_sec_ip),priority=90,match=(inport == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.src == fa:16:3e:5f:33:3f && ip4.src == {10.0.0.7}), action=(next;)

```

Egress ACL - Da perspectiva de uma VM, o tráfego de rede será de Egress e de acordo com o que o Neutron programou no Security Group, todo o tráfego de rede de Egress da VM estará liberado. Nesse estágio, a table 8 irá dar match em todas as novas conexões vindas da VM1 e irá marcá-las com o connection tracking. A próxima table irá verificar a porta em um Port Group e executará o commit do connection tracker:

```

1 table=8 (ls_in_acl_hint), priority=7, match=(ct.new && !ct.est), action=(reg0[7] = 1; reg0[9] = 1; next;)
2 table=9 (ls_in_acl),priority=2002,match=(reg0[7] == 1 && (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4)), action=(reg0[1] = 1; next;)
3 table=9 (ls_in_acl),priority=2002,match=(reg0[7] == 1 && (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6)), action=(reg0[1] = 1; next;)
4 table=9 (ls_in_acl),priority=2002,match=(reg0[8] == 1 && (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4)), action=(next;)
5 table=9 (ls_in_acl),priority=2002,match=(reg0[8] == 1 && (inport == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6)), action=(next;)

```

ARP Responder - Para a requisição de resolução ARP, será gerado a resposta fazendo a troca entre MAC Address de origem com o de destino:

```

1 table=18(ls_in_arp_rsp),priority=50,match=(arp.tpa == 10.0.0.1 && arp.op == 1), action=(eth.dst = eth.src; eth.src = fa:16:3e:8f:c6:8b; arp.op = 2; /*
2 table=18(ls_in_arp_rsp),priority=50,match=(arp.tpa == 10.0.0.2 && arp.op == 1), action=(eth.dst = eth.src; eth.src = fa:16:3e:3d:c6:9e; arp.op = 2; /*

```

```

3 table=18(ls_in_arp_rsp),priority=50,match=(arp.tpa == 10.0.0.42 && arp.op == 1), action=(eth.dst = eth.src; eth.src = fa:16:3e:77:12:96; arp.op = 2; /*
4 table=18(ls_in_arp_rsp),priority=50,match=(arp.tpa == 10.0.0.7 && arp.op == 1), action=(eth.dst = eth.src; eth.src = fa:16:3e:5f:33:3f; arp.op = 2; /*

```

DHCP Processing - Conforme já explicado anteriormente, o funcionamento do DHCP ocorre através da manipulação dos flows:

```

1 table=19(ls_in_dhcp_options),priority=100,match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4.src == 0.0.0.0
2 table=19(ls_in_dhcp_options),priority=100,match=(inport == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.src == fa:16:3e:5f:33:3f && ip4.src == 0.0.0.0
3 table=20(ls_in_dhcp_response),priority=100,match=(inport == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.src == fa:16:3e:77:12:96 && ip4 && udp.src ==
4 table=20(ls_in_dhcp_response),priority=100,match=(inport == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.src == fa:16:3e:5f:33:3f && ip4 && udp.src ==
5 table=20(ls_in_dhcp_response),priority=0,match=(1), action=(next;);

```

Destination Lookup - Verifica qual é o MAC Address de destino e determina a porta de output:

```

1 table=24(ls_in_l2_lookup),priority=50,match=(eth.dst == fa:16:3e:5f:33:3f), action=(output = "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc"; output;);
2 table=24(ls_in_l2_lookup),priority=50,match=(eth.dst == fa:16:3e:77:12:96), action=(output = "04a459b1-6ef0-45c2-aedc-063c63b716f9"; output;);

```

Ingress ACL - Implementa as rules de Ingress do Security Group do Neutron. Neste exemplo, existe uma rule que libera conexões de Ingress do SSH. O princípio, o funcionamento é o mesmo, pacotes que tem match em uma table são submetidos para outra table de connection tracking:

```

1 table=4 (ls_out_acl),priority=2002,match=(reg0[7] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == $pg_fed05084_5a95_4
2 table=4 (ls_out_acl),priority=2002,match=(reg0[7] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == 0.0.0.0/0 && tcp &&
3 table=4 (ls_out_acl),priority=2002,match=(reg0[7] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6 && ip6.src == $pg_fed05084_5a95_4
4 table=4 (ls_out_acl),priority=2002,match=(reg0[8] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == $pg_fed05084_5a95_4
5 table=4 (ls_out_acl),priority=2002,match=(reg0[8] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip4 && ip4.src == 0.0.0.0/0 && tcp &&
6 table=4 (ls_out_acl),priority=2002,match=(reg0[8] == 1 && (output == @pg_fed05084_5a95_4054_87e3_794fca4f7075 && ip6 && ip6.src == $pg_fed05084_5a95_4
7 table=7 (ls_out_stateful),priority=100,match=(reg0[1] == 1 && reg0[13] == 0), action=(ct_commit { ct_label.blocked = 0; }; next;);
8 table=7 (ls_out_stateful),priority=100,match=(reg0[1] == 1 && reg0[13] == 1), action=(ct_commit { ct_label.blocked = 0; ct_label.label = reg3; }; next;);
9 table=7 (ls_out_stateful),priority=0,match=(1), action=(next;);

```

Port Security - Verifica se os pacotes que estão ingressando na Switch Logical Port possuem os endereços de MAC Address e IP de destino corretos:

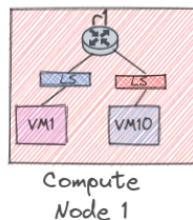
```

1 table=8 (ls_out_port_sec_ip),priority=90,match=(output == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.dst == fa:16:3e:77:12:96 && ip4.dst == {255.25
2 table=8 (ls_out_port_sec_ip),priority=90,match=(output == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.dst == fa:16:3e:5f:33:3f && ip4.dst == {255.25
3 table=9 (ls_out_port_sec_l2),priority=50,match=(output == "04a459b1-6ef0-45c2-aedc-063c63b716f9" && eth.dst == {fa:16:3e:77:12:96}), action=(output;);
4 table=9 (ls_out_port_sec_l2),priority=50,match=(output == "7e5871ed-6dd5-46bf-a30d-b02dfc3981dc" && eth.dst == {fa:16:3e:5f:33:3f}), action=(output;);

```

O próximo passo é fazer uma análise do tráfego L3 entre duas VMs que estão em diferentes subnets e entenderemos os Logical Flows instalados nos pipelines de Ingress e Egress.

Será necessário criar uma segunda subnet (private2), conectar no router existente e criar uma nova VM (VM10) nessa nova subnet. Os Logical Flows que teremos no final seguirá o diagrama abaixo:



```

1 # ovn-nbctl show f6b46d50-fbdc-495f-a195-0bf4a92be2be
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)
3   port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
4     mac: "fa:16:3e:71:b4:1f"
5     networks: ["172.24.4.147/24", "2001:db8::1/64"]
6     gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
7   port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
8     mac: "fa:16:3e:03:e6:64"
9     networks: ["fd42:4193:f777::1/64"]
10  port lrp-60a3de5f-f911-4222-8774-fb188c082166
11    mac: "fa:16:3e:cd:51:bf"
12    networks: ["10.0.0.65/26"]
13  port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
14    mac: "fa:16:3e:8f:c6:8b"
15    networks: ["10.0.0.1/26"]
16  nat 0485e6c0-0aa8-4cd7-b4a1-ce34b9875cfc
17    external ip: "172.24.4.147"
18    logical ip: "10.0.0.64/26"
19    type: "snat"
20  nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
21    external ip: "172.24.4.147"
22    logical ip: "10.0.0.0/26"
23    type: "snat"
24  nat 8db7a3ed-2897-41ce-8fa6-3a795e540a3b
25    external ip: "172.24.4.96"

```

```

26     logical ip: "10.0.0.42"
27     type: "dnat_and_snat"

```

Similar com o tráfego de rede L2 no mesmo Logical Switch, o tráfego de rede L3 entre Logical Switches também seguirá pelos pipelines de Ingress e Egress e tendo um Logical Router roteando os pacotes entre os Logical Switches.

Usaremos nesse exemplo as seguintes informações de cada VM:

- **VM1**
 - Switch Logical Port: 04a459b1-6ef0-45c2-aedc-063c63b716f9
 - MAC Address: fa:16:3e:77:12:96
 - IP Address: 10.0.0.42
- **VM10**
 - Switch Logical Port: 7cdf62e1-64a3-4a6a-9d78-96477b6e2c4c
 - MAC Address: fa:16:3e:e7:5a:17
 - IP Address: 10.0.0.67
- Logical Switch 1 (**private**): 4830c62f-7de6-4c6d-91aa-6eade80c7291
- Logical Switch 2 (**private2**): 463dd628-03ba-4e94-ad60-04343b3828e9
- Logical Router: f6b46d50-fbdc-495f-a195-0bf4a92be2be
- Logical Router Port (private): 286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
 - MAC Address: fa:16:3e:8f:c6:8b
- Logical Router Port (private2): 60a3de5f-f911-4222-8774-fb188c082166
 - MAC Address: fa:16:3e:cd:51:bf

É possível checar os Logical Flows de ambos Logical Switches conforme abaixo:

```

1 # ovn-sbctl lflow-list 4830c62f-7de6-4c6d-91aa-6eade80c7291
2 # ovn-sbctl lflow-list 463dd628-03ba-4e94-ad60-04343b3828e9

```

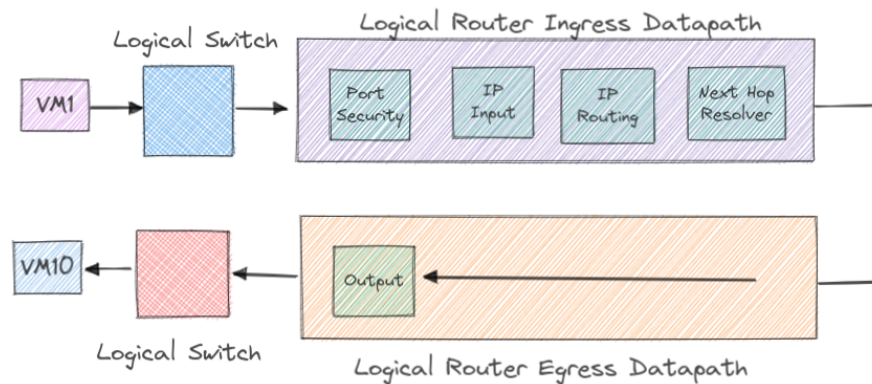
Para nossa análise do tráfego de rede L3, nos concentraremos nos pipelines de Ingress e Egress do Logical Router:

```

1 # ovn-sbctl lflow-list f6b46d50-fbdc-495f-a195-0bf4a92be2be

```

Podemos verificar o resumo dos estágios da conectividade leste-oeste de cada pipeline no diagrama abaixo:



Port Security - Faz uma checagem de segurança para os pacotes que estão ingressando na Logical Router Port:

```

1 table=0 (lr_in_admission),priority=50,match=(eth.dst == fa:16:3e:8f:c6:8b && inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"), action=(xreg0[0..47]
2 table=0 (lr_in_admission),priority=50,match=(eth.dst == fa:16:3e:cd:51:bf && inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166"), action=(xreg0[0..47]
3 table=0 (lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"), action=(xreg0[0..47] = fa:16:3e:8f:c6:
4 table=0 (lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166"), action=(xreg0[0..47] = fa:16:3e:cd:51:
5 table=0 (lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183"), action=(xreg0[0..47] = fa:16:3e:03:e6:
6 table=0 (lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1"), action=(xreg0[0..47] = fa:16:3e:71:b4:

```

IP Input - Executa algumas checagens no nível de L3 e implementa no Logical Router algumas manipulações com flows para o mesmo responder a ICMP e ARP Reply:

```

1 table=3 (lr_in_ip_input),priority=100,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && ip4 && ip.ttl == {0, 1} && !ip.later_frag), action
2 table=3 (lr_in_ip_input),priority=100,match=(inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166" && ip4 && ip.ttl == {0, 1} && !ip.later_frag), action
3 table=3 (lr_in_ip_input),priority=90,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && arp.op == 1 && arp.tpa == 10.0.0.1 && arp.spa == 10
4 table=3 (lr_in_ip_input),priority=90,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && ip6.dst == {fe80::f816:3eff:fe8f:c68b, ff02::1:ff8f
5 table=3 (lr_in_ip_input),priority=90,match=(inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166" && arp.op == 1 && arp.tpa == 10.0.0.65 && arp.spa == 1

```

IP Routing - Implementa o roteamento baseado no endereço IP de destino. Nesse estágio, é decidido a porta de output, TTL do pacote é decrementado e o next hop é definido:

```

1 table=11(lr_in_ip_routing),priority=80,match=(ip4.dst == 10.0.0.0/26), action=(ip.ttl--; reg8[0..15] = 0; reg0 = ip4.dst; reg1 = 10.0.0.1; eth.src = fa
2 table=11(lr_in_ip_routing),priority=80,match=(ip4.dst == 10.0.0.64/26), action=(ip.ttl--; reg8[0..15] = 0; reg0 = ip4.dst; reg1 = 10.0.0.65; eth.src = fa

```

Next Hop Resolver - Realiza o discover do MAC Address do IP de destino para entregar o pacote. Nesse caso, para o tráfego de rede leste-oeste, os MAC Addresses já são conhecidos pelo OVN Southbound e portanto existirão flows definidos:

```
1 table=15(lr_in_arp_resolve),priority=100,match=(output == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && reg0 == 10.0.0.42), action=(eth.dst = fa:16:3e:03:e6:64)
2 table=15(lr_in_arp_resolve),priority=100,match=(output == "lrp-60a3de5f-f911-4222-8774-fb188c082166" && reg0 == 10.0.0.67), action=(eth.dst = fa:16:3e:03:e6:64)
```

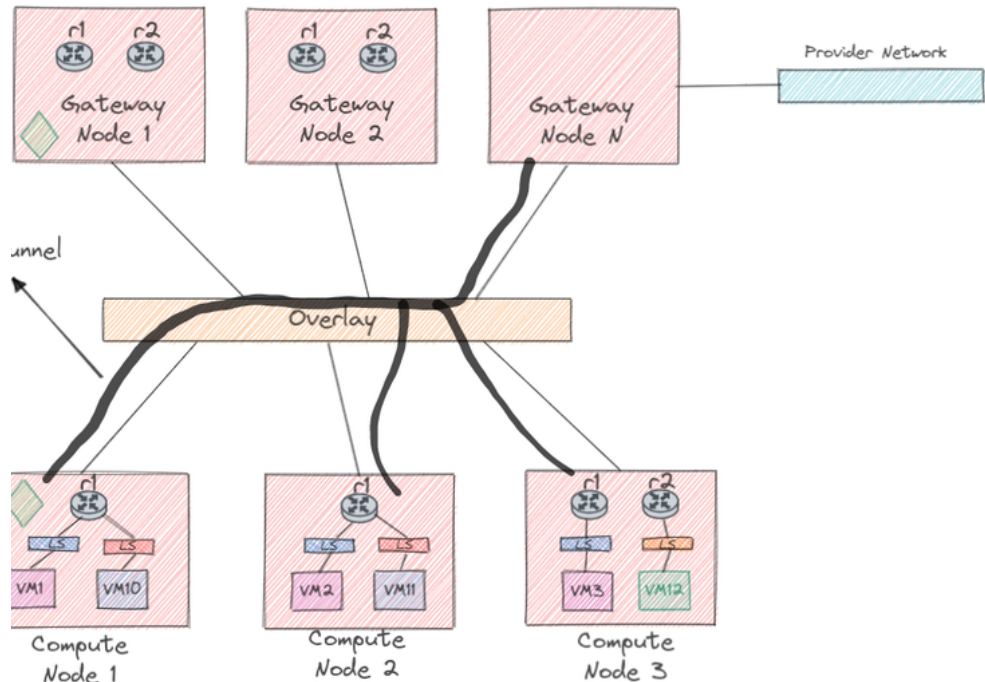
Output - Envia o pacote para a porta de output definida durante o estágio do IP Routing:

```
1 table=6(lr_out_delivery),priority=100,match=(output == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"), action=(output;)
2 table=6(lr_out_delivery),priority=100,match=(output == "lrp-60a3de5f-f911-4222-8774-fb188c082166"), action=(output;)
```

Packet Walkthrough - Conectividade norte-sul

A conectividade norte-sul se resume na comunicação de rede entre workloads internos com ambientes externos e vice versa.

Conforme o diagrama abaixo está descrevendo, os Logical Routers são distribuídos entre os Chassis e são responsáveis pelo roteamento e NAT de acordo com o tipo do Chassi:



Em nossa arquitetura, o Gateway Node (Chassi Gateway) é responsável em criar conectividade com redes externas e o mesmo irá iniciar/terminar os túneis Geneve para permitir o tráfego de rede norte-sul saia e entre.

Os Logical Routers possuem regras de NAT para implementar o DNAT (1:1) e SNAT (1: para muitos). Do ponto de vista do OVN Northbound, podemos visualizar as portas criadas de um Logical Router e também as regras de DNAT e SNAT:

```
1 # ovn-nbctl show f6b46d50-fbdc-495f-a195-0bf4a92be2be
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)
3   port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
4     mac: "fa:16:3e:71:b4:1f"
5     networks: ["172.24.4.147/24", "2001:db8::1/64"]
6     gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
7   port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
8     mac: "fa:16:3e:03:e6:64"
9     networks: ["fd42:4193:f777::1/64"]
10  port lrp-60a3de5f-f911-4222-8774-fb188c082166
11    mac: "fa:16:3e:cd:51:bf"
12    networks: ["10.0.0.65/26"]
13  port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
14    mac: "fa:16:3e:8f:c6:8b"
15    networks: ["10.0.0.1/26"]
16  nat 0485e6c0-0aa8-4cd7-b4a1-ce34b9875cfc
17    external ip: "172.24.4.147"
18    logical ip: "10.0.0.64/26"
19    type: "snat"
20  nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
21    external ip: "172.24.4.147"
22    logical ip: "10.0.0.0/26"
23    type: "snat"
```

```

24 nat 8db7a3ed-2897-41ce-8fa6-3a795e540a3b
25     external ip: "172.24.4.96"
26     logical ip: "10.0.0.42"
27     type: "dnat_and_snat"

```

Nos output acima, existe o campo external ip que é o endereço IP utilizado para o DNAT ou SNAT.

Usaremos nesse exemplo as seguintes informações de cada VM:

- **VM1**
 - Switch Logical Port: 04a459b1-6ef0-45c2-aedc-063c63b716f9
 - MAC Address: fa:16:3e:77:12:96
 - IP Address: 10.0.0.42
- Logical Switch 1 (**private**): 4830c62f-7de6-4c6d-91aa-6eade80c7291
- Logical Router: f6b46d50-fbdc-495f-a195-0bf4a92be2be
- Logical Router Port (private): 286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
 - MAC Address: fa:16:3e:8f:c6:8b
- Logical Router Port (externa): cf47f4af-5cfd-4293-8254-1573856157c1
 - IP externo SNAT: 172.24.4.147
 - IP externo DNAT: 172.24.4.96

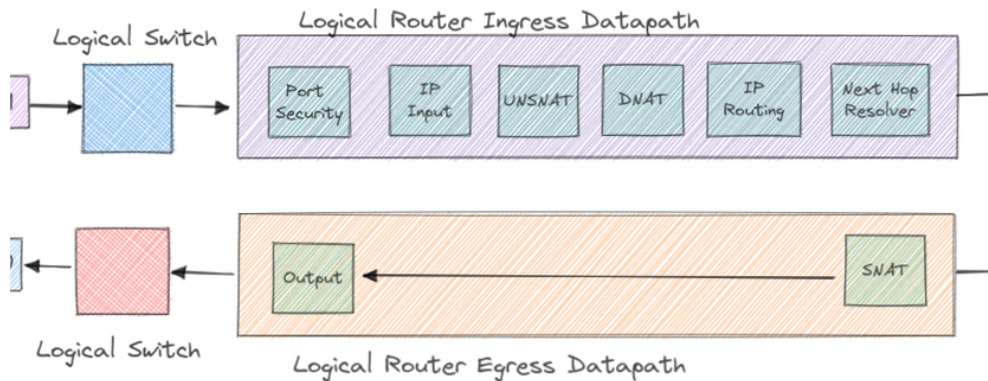
Para nossa análise do tráfego de rede L3 norte-sul, nos concentraremos nos pipelines de Ingress e Egress do Logical Router:

```

1 # ovn-sbctl lflow-list f6b46d50-fbdc-495f-a195-0bf4a92be2be

```

Podemos verificar o resumo dos estágios da conectividade norte-sul de cada pipeline no diagrama abaixo:



Port Security - Faz uma checagem de segurança para os pacotes que estão ingressando na Logical Router Port:

```

1 table=0(lr_in_admission),priority=50,match=(eth.dst == fa:16:3e:8f:c6:8b && inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"), action=(xreg0[0..47]
2 table=0(lr_in_admission),priority=50,match=(eth.dst == fa:16:3e:cd:51:bf && inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166"), action=(xreg0[0..47]
3 table=0(lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0"), action=(xreg0[0..47] = fa:16:3e:8f:c6:8b
4 table=0(lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166"), action=(xreg0[0..47] = fa:16:3e:cd:51:bf
5 table=0(lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183"), action=(xreg0[0..47] = fa:16:3e:03:e6:6b
6 table=0(lr_in_admission),priority=50,match=(eth.mcast && inport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1"), action=(xreg0[0..47] = fa:16:3e:71:b4:11

```

IP Input - Executa algumas checagens no nível de L3 e implementa no Logical Router várias manipulações com flows para o mesmo responder a ICMP e ARP Reply:

```

1 table=3(lr_in_ip_input),priority=100,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && ip4 && ip.ttl == {0, 1} && !ip.later_frag), action=
2 table=3(lr_in_ip_input),priority=100,match=(inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166" && ip4 && ip.ttl == {0, 1} && !ip.later_frag), action=
3 table=3(lr_in_ip_input),priority=90,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && arp.op == 1 && arp.tpa == 10.0.0.1 && arp.spa == 10.
4 table=3(lr_in_ip_input),priority=90,match=(inport == "lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0" && ip6.dst == {fe80::f816:3eff:fe8f:c68b, ff02::1:ff8f:
5 table=3(lr_in_ip_input),priority=90,match=(inport == "lrp-60a3de5f-f911-4222-8774-fb188c082166" && arp.op == 1 && arp.tpa == 10.0.0.65 && arp.spa == 10.

```

UNSNAT - Faz a tradução do endereço de IP de destino para o endereço real interno para o tráfego de rede vindo de uma rede externa:

```

1 table=4(lr_in_unsnat),priority=100,match=(ip && ip4.dst == 172.24.4.147 && inport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && flags.loopback == 1

```

DNAT - Implementa o NAT 1:1 do IP externo para um IP interno:

```

1 table=6(lr_in_dnat),priority=100,match=(ip && ip4.dst == 172.24.4.96 && inport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && is_chassis_resident("cr

```

IP Routing - Implementa o roteamento baseado no endereço IP de destino. Nesse estágio, é decidido a porta de output, TTL do pacote é decrementado e o next hop é definido:

```

1 table=11(lr_in_ip_routing),priority=74,match=(ip4.dst == 172.24.4.0/24), action=(ip.ttl--; reg8[0..15] = 0; reg0 = ip4.dst; reg1 = 172.24.4.147; eth.sr
2 table=11(lr_in_ip_routing),priority=1,match=(reg7 == 0 && ip4.dst == 0.0.0.0/0), action=(ip.ttl--; reg8[0..15] = 0; reg0 = 172.24.4.1; reg1 = 172.24.4.

```

Next Hop Resolver - Realiza o discover do MAC Address do IP de destino para entregar o pacote. Nesse caso, para o tráfego de rede norte-sul, os MAC Addresses externos aprendidos serão armazenados na table **MAC_Binding** do Southbound database:

```
1 table=15(lr_in_arp_resolve),priority=100,match=(outport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && reg0 == 172.24.4.147), action=(eth.dst = fa:16:3e:77:12:96)
2 table=15(lr_in_arp_resolve),priority=100,match=(outport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && reg0 == 172.24.4.96), action=(eth.dst = fa:16:3e:77:12:96)
```

SNAT - Implementa o SNAT, traduzindo o IP de origem e portas TCP/UDP de uma subnet para um endereço IP externo:

```
1 table=3 (lr_out_snat),priority=156,match=(ip && ip4.src == 10.0.0.0/26 && outport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && is_chassis_resident(
2 table=3 (lr_out_snat),priority=155,match=(ip && ip4.src == 10.0.0.64/26 && outport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1" && is_chassis_resident
```

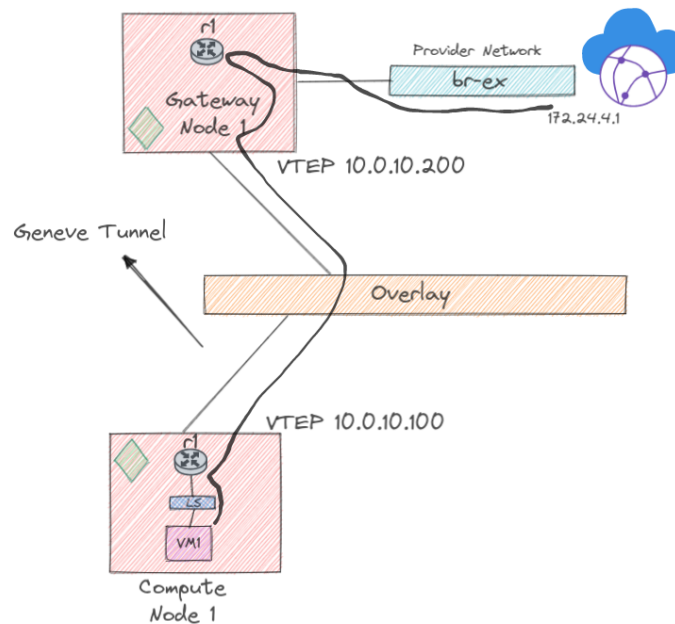
Output - Envia o pacote para a porta de output definida durante o estágio do IP Routing:

```
1 table=6(lr_out_delivery),priority=100,match=(outport == "lrp-cf47f4af-5cfd-4293-8254-1573856157c1"), action=(output;)
```

Openflow Walkthrough [↗](#)

Nesse tópico faremos uma análise dos Openflow flows no nível dos Chassis que foram instalados no OVS a partir dos Logical Flows definidos no OVN Central.

Para demonstrar os detalhes dos Openflow flows implementados, iremos analisar o fluxo de comunicação entre a **vm1** com o IP externo **172.24.4.1** conforme o diagrama abaixo:



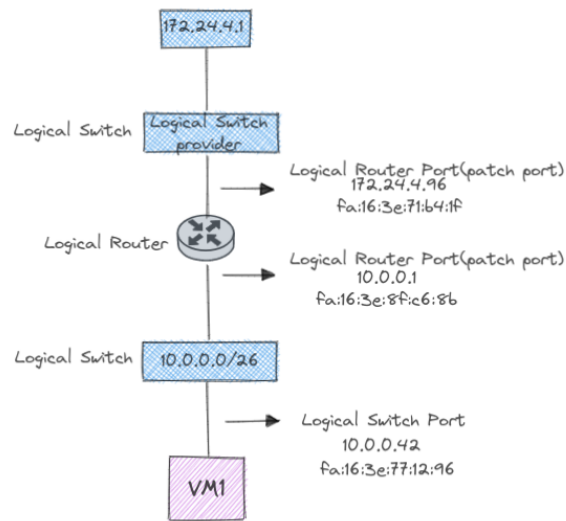
Faremos um trace e simularemos a comunicação via ICMP da VM1 e teremos que identificar o ID da porta conforme abaixo, verifique utilizando os 3 últimos campos do MAC Address:

```
1 # ovs-ofctl show br-int | grep 77:12:96
2 3(tap04a459b1-6e): addr:fe:16:3e:77:12:96
```

Teremos que identificar o MAC Address do gateway da subnet onde a VM1 está conectada, em resumo vamos precisar das seguintes informações:

- dl_src - MAC Address da VM1
- dl_dst - MAC Address do gateway da subnet (private)
- nw_src - Endereço IP da VM1
- nw_dst - Endereço IP de destino

Conforme o diagrama abaixo, os Logical Flows criados no OVN Northbound serão traduzidos para os Openflow flows:



Para identificar os endereços IP e MAC Address das interfaces do Logical Router, execute o seguinte comando trocando o ID pelo do seu router:

```
1 # ovn-nbctl show f6b46d50-fbdc-495f-a195-0bf4a92be2be
2 router f6b46d50-fbdc-495f-a195-0bf4a92be2be (neutron-d18829fb-2cf8-47b7-85d6-0e96caf2e90f) (aka router1)
3   port lrp-cf47f4af-5cfd-4293-8254-1573856157c1
4     mac: "fa:16:3e:71:b4:1f"
5     networks: ["172.24.4.147/24", "2001:db8::1/64"]
6     gateway chassis: [0727afaf-5048-4624-a898-1181367f0233]
7   port lrp-7bbb8967-b1dd-475e-b9ec-e6aba3931183
8     mac: "fa:16:3e:03:e6:64"
9     networks: ["fd42:4193:f777::1/64"]
10  port lrp-60a3de5f-f911-4222-8774-fb188c082166
11    mac: "fa:16:3e:cd:51:bf"
12    networks: ["10.0.0.65/26"]
13  port lrp-286e2fc1-eb1c-46b2-bf9b-d409be3c62b0
14    mac: "fa:16:3e:8f:c6:8b"
15    networks: ["10.0.0.1/26"]
16  nat 0485e6c0-0aa8-4cd7-b4a1-ce34b9875cfc
17    external ip: "172.24.4.147"
18    logical ip: "10.0.0.64/26"
19    type: "snat"
20  nat 46e4c3d0-4154-4b39-8b2d-8cd10bc8dc6b
21    external ip: "172.24.4.147"
22    logical ip: "10.0.0.0/26"
23    type: "snat"
24  nat 8db7a3ed-2897-41ce-8fa6-3a795e540a3b
25    external ip: "172.24.4.96"
26    logical ip: "10.0.0.42"
27    type: "dnat_and_snat"
```

Vamos executar o trace substituindo os valores das opções conforme seu setup:

```
1 # ovs-appctl ofproto/trace br-int in_port=3,icmp,\
2 d1_src=fa:16:3e:77:12:96,d1_dst=fa:16:3e:8f:c6:8b,nw_src=10.0.0.42,\
3 nw_dst=172.24.4.1,nw_ttl=64
4 Flow: icmp,in_port=3,vlan_tci=0x0000,d1_src=fa:16:3e:77:12:96,d1_dst=fa:16:3e:8f:c6:8b,nw_src=10.0.0.42,nw_dst=172.24.4.1,nw_tos=0,nw_ecn=0,nw_ttl=64
5
6 bridge("br-int")
7 -----
8 0. in_port=3, priority 100, cookie 0xae5a82a2
9   set_field:0x3->reg13
10  set_field:0x5->reg11
11  set_field:0x4->reg12
12  set_field:0x1->metadata
13  set_field:0x4->reg14
14  resubmit(,8)
15 8. reg14=0x4,metadata=0x1,d1_src=fa:16:3e:77:12:96, priority 50, cookie 0xc8a2b498
16   resubmit(,9)
17 9. ip,reg14=0x4,metadata=0x1,d1_src=fa:16:3e:77:12:96,nw_src=10.0.0.42, priority 90, cookie 0xabeb0292
18   resubmit(,10)
19 10. metadata=0x1, priority 0, cookie 0xfe84b4d2
20   resubmit(,11)
21 11. metadata=0x1, priority 0, cookie 0x18b31b1e
```

```

22     resubmit(,12)
23 12. metadata=0x1, priority 0, cookie 0xf3ee973
24     resubmit(,13)
25 13. ip,metadata=0x1, priority 100, cookie 0x206dc7b4
26     set_field:0x10000000000000000000000000000000/0x1000000000000000000000000->xxreg0
27     resubmit(,14)
28 14. metadata=0x1, priority 0, cookie 0x805ac6e5
29     resubmit(,15)
30 15. ip,reg0=0x1/0x1,metadata=0x1, priority 100, cookie 0xd258468c
31     ct(table=16,zone=NXM_NX_REG13[0..15])
32     drop
33     -> A clone of the packet is forked to recirculate. The forked pipeline will be resumed at table 16.
34     -> Sets the packet to an untracked state, and clears all the conntrack fields.
35
36 Final flow: icmp,reg0=0x1,reg11=0x5,reg12=0x4,reg13=0x3,reg14=0x4,metadata=0x1,in_port=3,vlan_tci=0x0000,dl_src=fa:16:3e:77:12:96,dl_dst=fa:16:3e:8f:
37 Megaflow: recirc_id=0,eth,icmp,in_port=3,dl_src=fa:16:3e:77:12:96,dl_dst=fa:16:3e:8f:c6:8b,nw_src=10.0.0.42,nw_frag=no
38 Datapath actions: ct(zone=3),recirc(0x102)
39
40 =====
41 recirc(0x102) - resume conntrack with default ct_state=trk|new (use --ct-next to customize)
42 =====
43
44 Flow: recirc_id=0x102,ct_state=new|trk,ct_zone=3,eth,icmp,reg0=0x1,reg11=0x5,reg12=0x4,reg13=0x3,reg14=0x4,metadata=0x1,in_port=3,vlan_tci=0x0000,dl_
45
46 bridge("br-int")
47 -----
48     thaw
49         Resuming from table 16
50 16. ct_state=new-est+trk,metadata=0x1, priority 7, cookie 0x6406b892
51     set_field:0x80000000000000000000000000000000/0x80000000000000000000000000000000->xxreg0
52     set_field:0x20000000000000000000000000000000/0x20000000000000000000000000000000->xxreg0
53     resubmit(,17)
54 17. ip,reg0=0x80/0x80,reg14=0x4,metadata=0x1, priority 2002, cookie 0x6920cfa0
55     set_field:0x20000000000000000000000000000000/0x20000000000000000000000000000000->xxreg0
56     resubmit(,18)
57 18. metadata=0x1, priority 0, cookie 0xa03b435a
58     resubmit(,19)
59 19. metadata=0x1, priority 0, cookie 0xfa1b9bd4
60     resubmit(,20)
61 20. metadata=0x1, priority 0, cookie 0xf599de52
62     resubmit(,21)
63 21. metadata=0x1, priority 0, cookie 0x9e7b797c
64     resubmit(,22)
65 22. ip,reg0=0x2/0x2002,metadata=0x1, priority 100, cookie 0x6039f3f1
66     ct(commit,zone=NXM_NX_REG13[0..15],nat(src),exec(set_field:0/0x1->ct_label))
67     nat(src)
68     set_field:0/0x1->ct_label
69     -> Sets the packet to an untracked state, and clears all the conntrack fields.
70     resubmit(,23)
71 23. metadata=0x1, priority 0, cookie 0x5404443f
72     resubmit(,24)
73 24. metadata=0x1, priority 0, cookie 0x9834a899
74     resubmit(,25)
75 25. metadata=0x1, priority 0, cookie 0x8df998df
76     resubmit(,26)
77 26. metadata=0x1, priority 0, cookie 0xe2b0942
78     resubmit(,27)
79 27. metadata=0x1, priority 0, cookie 0x7abf55e8
80     resubmit(,28)
81 28. metadata=0x1, priority 0, cookie 0xbc195589
82     resubmit(,29)
83 29. metadata=0x1, priority 0, cookie 0x3a7af485
84     resubmit(,30)
85 30. metadata=0x1, priority 0, cookie 0xe6f6515c
86     resubmit(,31)
87 31. metadata=0x1, priority 0, cookie 0xeefb9688
88     resubmit(,32)
89 32. metadata=0x1,dl_dst=fa:16:3e:8f:c6:8b, priority 50, cookie 0x18e68560
90     set_field:0x2->reg15
91     resubmit(,37)
92 37. priority 0
93     resubmit(,38)
94 38. reg15=0x2,metadata=0x1, priority 100, cookie 0x319c3195
95     set_field:0x5->reg11
96     set_field:0x4->reg12
97     resubmit(,39)

```

```
98 39. priority 0
99   set_field:0->reg0
100  set_field:0->reg1
101  set_field:0->reg2
102  set_field:0->reg3
103  set_field:0->reg4
104  set_field:0->reg5
105  set_field:0->reg6
106  set_field:0->reg7
107  set_field:0->reg8
108  set_field:0->reg9
109  resubmit(,40)
110 40. ip,reg15=0x2,metadata=0x1, priority 110, cookie 0x86c69411
111   resubmit(,41)
112 41. ip,reg15=0x2,metadata=0x1, priority 110, cookie 0x8cd25afb
113   resubmit(,42)
114 42. metadata=0x1, priority 0, cookie 0x6093622
115   resubmit(,43)
116 43. ct_state=-trk,metadata=0x1, priority 5, cookie 0xa3201784
117   set_field:0x100000000000000000000000000000000/0x10000000000000000000000000->xxreg0
118   set_field:0x200000000000000000000000000000000/0x20000000000000000000000000->xxreg0
119   resubmit(,44)
120 44. metadata=0x1, priority 0, cookie 0xbc300582
121   resubmit(,45)
122 45. metadata=0x1, priority 0, cookie 0xf77e24b3
123   resubmit(,46)
124 46. metadata=0x1, priority 0, cookie 0x88b3f8c6
125   resubmit(,47)
126 47. metadata=0x1, priority 0, cookie 0x3dd5a45d
127   resubmit(,48)
128 48. metadata=0x1, priority 0, cookie 0x8cc5f659
129   resubmit(,49)
130 49. reg15=0x2,metadata=0x1, priority 50, cookie 0x366d5134
131   resubmit(,64)
132 64. priority 0
133   resubmit(,65)
134 65. reg15=0x2,metadata=0x1, priority 100, cookie 0x319c3195
135   clone(ct_clear,set_field:0->reg11,set_field:0->reg12,set_field:0->reg13,set_field:0x2->reg11,set_field:0x1->reg12,set_field:0x2->metadata,set_fie
136   ct_clear
137   set_field:0->reg11
138   set_field:0->reg12
139   set_field:0->reg13
140   set_field:0x2->reg11
141   set_field:0x1->reg12
142   set_field:0x2->metadata
143   set_field:0x1->reg14
144   set_field:0->reg10
145   set_field:0->reg15
146   set_field:0->reg0
147   set_field:0->reg1
148   set_field:0->reg2
149   set_field:0->reg3
150   set_field:0->reg4
151   set_field:0->reg5
152   set_field:0->reg6
153   set_field:0->reg7
154   set_field:0->reg8
155   set_field:0->reg9
156   resubmit(,8)
157 8. reg14=0x1,metadata=0x2,d1_dst=fa:16:3e:8f:c6:8b, priority 50, cookie 0x5dd1c1da
158   set_field:0xfa163e8fc68b00000000000000000000/0xfffffffffff00000000000000000->xxreg0
159   resubmit(,9)
160 9. metadata=0x2, priority 0, cookie 0xdb1e5414
161   set_field:0x4/0x4->xreg4
162   resubmit(,10)
163 10. reg9=0/0x8,metadata=0x2, priority 100, cookie 0x14d8223b
164   resubmit(,11)
165 11. metadata=0x2, priority 0, cookie 0x75907ac0
166   resubmit(,12)
167 12. metadata=0x2, priority 0, cookie 0x75f9267d
168   resubmit(,13)
169 13. metadata=0x2, priority 0, cookie 0x6795240
170   resubmit(,14)
171 14. metadata=0x2, priority 0, cookie 0x230a1801
172   resubmit(,15)
173 15. metadata=0x2, priority 0, cookie 0xdff1c05a
```

```

174     resubmit(,16)
175 16. metadata=0x2, priority 0, cookie 0x75871c8
176     resubmit(,17)
177 17. metadata=0x2, priority 0, cookie 0xf9482e15
178     resubmit(,18)
179 18. metadata=0x2, priority 0, cookie 0x4f5fe2d5
180     set_field:0/0xffffffff->xxreg1
181     resubmit(,19)
182 19. ip,metadata=0x2,nw_dst=172.24.4.0/24, priority 74, cookie 0x89863b5d
183     dec_ttl()
184     set_field:0/0xffff00000000->xreg4
185     move:NXM_OF_IP_DST[]->NXM_NX_XXREG0[96..127]
186     -> NXM_NX_XXREG0[96..127] is now 0xac180401
187     set_field:0xac1804930000000000000000/0xffffffff0000000000000000->xxreg0
188     set_field:fa:16:3e:71:b4:1f->eth_src
189     set_field:0x2->reg15
190     set_field:0x1/0x1->reg10
191     resubmit(,20)
192 20. reg8=0/0xffff,metadata=0x2, priority 150, cookie 0x4316ad8e
193     resubmit(,21)
194 21. metadata=0x2, priority 0, cookie 0x594cbd16
195     set_field:0/0xffff00000000->xreg4
196     resubmit(,22)
197 22. reg8=0/0xffff,metadata=0x2, priority 150, cookie 0xa0cacfec
198     resubmit(,23)
199 23. ip,metadata=0x2, priority 0, cookie 0x7efd30e3
200     push:NXM_NX_REG0[]
201     push:NXM_NX_XXREG0[96..127]
202     pop:NXM_NX_REG0[]
203     -> NXM_NX_REG0[] is now 0xac180401
204     set_field:00:00:00:00:00:00->eth_dst
205     resubmit(,66)
206 66. reg0=0xac180401,reg15=0x2,metadata=0x2, priority 100, cookie 0xf880e921
207     set_field:0e:d5:cd:6a:ae:43->eth_dst
208     set_field:0x40/0x40->reg10
209     pop:NXM_NX_REG0[]
210     -> NXM_NX_REG0[] is now 0xac180401
211     resubmit(,24)
212 24. metadata=0x2, priority 0, cookie 0xdb31dbbf
213     resubmit(,25)
214 25. metadata=0x2, priority 0, cookie 0x93f6dedc
215     resubmit(,26)
216 26. reg15=0x2,metadata=0x2, priority 50, cookie 0xba7d6118
217     set_field:0x3->reg15
218     resubmit(,27)
219 27. metadata=0x2, priority 0, cookie 0x2af3587a
220     resubmit(,37)
221 37. reg15=0x3,metadata=0x2, priority 100, cookie 0xac1919ac
222     set_field:0x2/0xffffffff->tun_id
223     set_field:0x3->tun_metadata0
224     move:NXM_NX_REG14[0..14]->NXM_NX_TUN_METADATA0[16..30]
225     -> NXM_NX_TUN_METADATA0[16..30] is now 0x1
226     output:2
227     -> output to kernel tunnel
228
229 Final flow: recirc_id=0x102,eth,icmp,reg0=0x300,reg11=0x5,reg12=0x4,reg13=0x3,reg14=0x4,reg15=0x2,metadata=0x1,in_port=3,vlan_tci=0x0000,d1_src=fa:16:3e:77:11:22
230 MegafLOW: recirc_id=0x102,ct_state=new-est-rel-rpl-inv+trk,ct_label=0/0x1,eth,icmp,tun_id=0/0xffffffff,tun_metadata0=NP,in_port=3,d1_src=fa:16:3e:77:11:22
231 Datapath actions: ct(commit,zone=3,label=0/0x1,nat(src)),set(tunnel(tun_id=0x2,dst=10.0.10.200,ttl=64,tp_dst=6081,geneve({class=0x102,type=0x80,len=4,cookie=0x102}))

```

A parte que mais nos interessa do output acima é o do Datapath actions:

```

1  Datapath actions: ct(commit,zone=3,label=0/0x1,nat(src)),set(tunnel(tun_id=0x2,dst=10.0.10.200,ttl=64,tp_dst=6081,geneve({class=0x102,type=0x80,len=4,cookie=0x102}))

```

Após percorrer as várias Openflow tables, o pacote será encaminhado via túnel Geneve para o Gateway Node que possui o IP de destino 10.0.10.200 no Underlay.

No Gateway Node, podemos checar qual a interface de túnel Geneve que o mesmo possui com o Compute Node de origem:

```

1  # ovs-vsctl show
2  4f81f863-0e84-4ae6-ad08-010af9855e90
3      Manager "ptcp:6640:127.0.0.1"
4      Bridge br-ex
5          Port br-ex
6              Interface br-ex
7                  type: internal
8          Port patch-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46-to-br-int
9              Interface patch-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46-to-br-int
10                 type: patch

```

```

11         options: {peer=patch-br-int-to-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46}
12     Bridge br-int
13         fail_mode: secure
14         datapath_type: system
15         Port ovn-cab023-0
16             Interface ovn-cab023-0
17                 type: geneve
18                 options: {csum="true", key=flow, remote_ip="10.0.10.100"}
19         Port patch-br-int-to-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46
20             Interface patch-br-int-to-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46
21                 type: patch
22                 options: {peer=patch-provnet-f62f29e7-afc6-467a-97b0-1d3988477b46-to-br-int}
23         Port br-int
24             Interface br-int
25                 type: internal
26     ovs_version: "2.17.0"

```

Conforme o output acima, a interface `ovn-cab023-0` é o túnel Geneve com o Compute Node. Podemos verificar o ID da porta no OVS da seguinte forma:

```

1 # ovs-ofctl show br-int
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:0000ea9b5a37bab6
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw_tos mod_tp_src mod_tp_dst
6 1(ovn-cab023-0): addr:32:c5:95:5b:a8:01
7     config: 0
8     state: 0
9     speed: 0 Mbps now, 0 Mbps max
10 2(patch-br-int-to): addr:da:d2:2f:12:e0:3a
11     config: 0
12     state: 0
13     speed: 0 Mbps now, 0 Mbps max
14 LOCAL(br-int): addr:ea:9b:5a:37:ba:b6
15     config: PORT_DOWN
16     state: LINK_DOWN
17     speed: 0 Mbps now, 0 Mbps max
18 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

```

Podemos concluir que o ID da porta `ovn-cab023-0` do túnel é o `1`, agora é possível; verificar os Openflow flows vinculados com essa porta de túnel:

```

1 # ovs-ofctl dump-flows br-int | grep in_port=1
2 cookie=0x0, duration=560521.717s, table=0, n_packets=344304, n_bytes=33686061, idle_age=8181, hard_age=65534, priority=100,in_port=1 actions=move:NXM_

```

O ID 2 de porta (`patch-br-int-to`) faz referência com a bridge `br-ex` presente no Gateway Node que é a Network Provider. Podemos checar na table 65 a existência dessa porta de output, essa table é responsável em direcionar os pacotes para a rede externa:

```

1 # ovs-ofctl dump-flows br-int | grep output:2
2 cookie=0x9d926420, duration=559802.358s, table=65, n_packets=352337, n_bytes=33981879, idle_age=8334, hard_age=65534, priority=100,reg15=0x1,metadata=

```

Nesse momento, faremos o trace ao contrário para verificar a volta do pacote utilizando a porta 2 como `in_port(br-ex)`:

```

1 # ovs-appctl ofproto/trace br-int in_port=2,icmp,nw_src=172.24.4.1,nw_dst=172.24.4.96,dl_dst=fa:16:3e:71:b4:1f,nw_ttl=64 | grep Datapath
2 Datapath actions: set(tunnel(tun_id=0x1,dst=10.0.10.100,ttl=64,tp_dst=6081,geneve({class=0x102,type=0x80,len=4,0x20004})),flags(df|csum|key)),set(eth(s

```

Conforme é possível verificar, o pacote será encaminhado via túnel Geneve para o Compute Node que possui o IP de destino 10.0.10.100 no Underlay.

Conclusão

Neste treinamento abordamos de forma teórica e prática o caminho dos pacotes nas Logical Flow tables do OVN e tivemos a chance de analisar via trace o caminho dos pacotes nas várias Openflow tables.

 Para dar sequência no próximo treinamento, não remova as configurações desse setup.